Реализация операций над заявками.

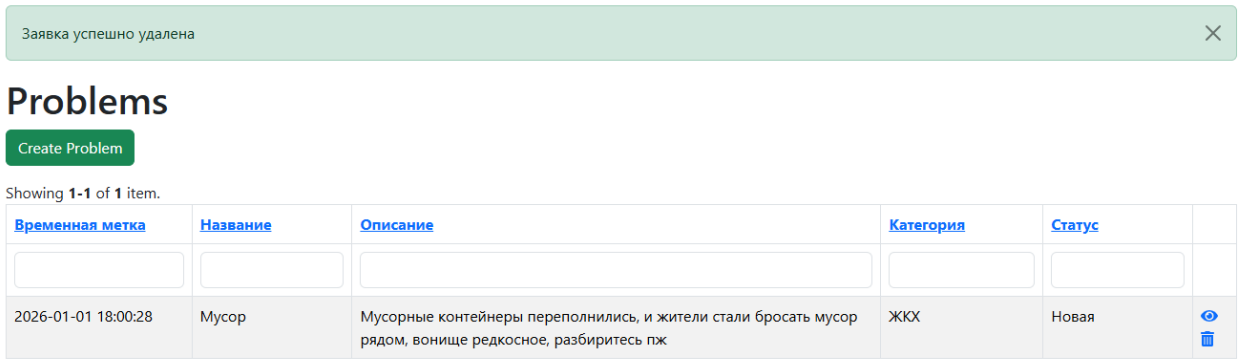
Первым этапом будет реализация удаление заявки пользователем. Но удаление будет выполнено только в том случае если статус равен “Новая”.

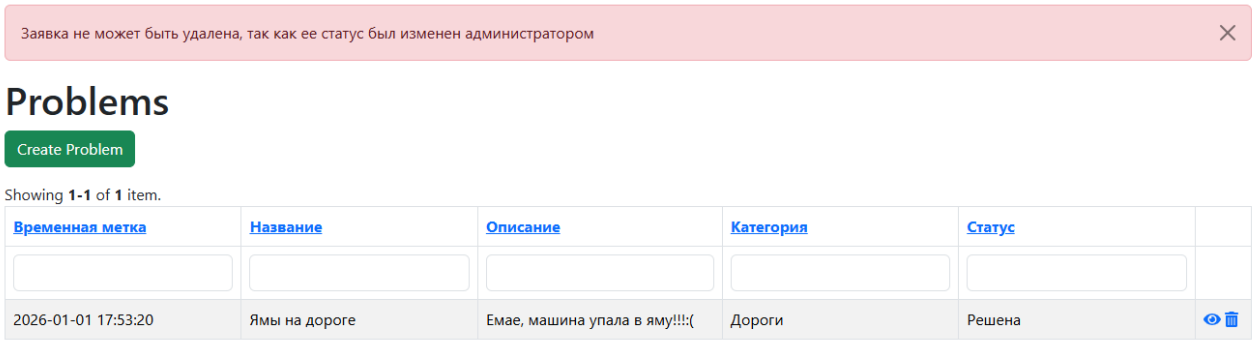
В файле LkController.php каталога controllers внутри функции actionDelete() выполним проверку на статус заявки. В случае если условия удовлетворительно, то мы удалим запись.

Также с помощью флешей выведем небольшой диалог через метод setFlash. Первый параметр — это класс, второй – сообщение.

```
if ( $this->findModel($id_problem)->status_problem == 'Новая') {
    $this->findModel($id_problem)->delete();
    Yii::$app->session->setFlash('success', 'Заявка успешно удалена');
} else {
    Yii::$app->session->setFlash('danger', 'Заявка не может быть удалена, так как ее статус был изменен администратором');
}
```

Реализация удаления.

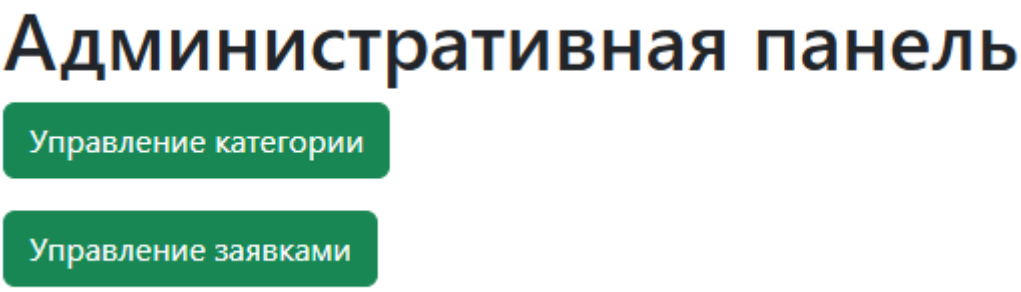




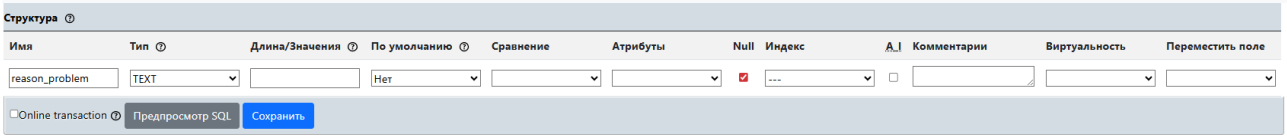
В файле index.php подкаталога admin каталога views добавим ещё одну ссылку на страницу с заявками.

```
<p>
    <?= Html::a('Управление заявками', ['/problem'], ['class' => 'btn btn-success']) ?>
</p>
```

Таким образом административная панель выглядит следующим образом.



В phpMyAdmin в таблице problem создадим поле reason_problem с типом данных полноценный текст, может быть нулевым.



В файле index.php подкаталога problem отобразим необходимые поля для вывода.

```
<?= GridView::widget([
    'dataProvider' => $dataProvider,
    'filterModel' => $searchModel,
    'columns' => [
        ['class' => 'yii\grid\SerialColumn'],

        // 'id_problem',
        'name_problem',
        'description_problem:ntext',
        'timestamp_problem',
        'id_user',
        //'id_category',
        'status_problem',
        //'photo_before',
        //'photo_after',
        [
            'class' => ActionColumn::className(),
            'urlCreator' => function ($action, Problem $model, $key, $index, $column) {
                return Url::toRoute([$action, 'id_problem' => $model->id_problem]);
            }
        ],
    ],
]); ?>
```

Оставим временную метку, название и описание заявки, ФИО пользователя, название категории и статус заявки.

```
// 'id_problem',
'timestamp_problem',
'name_problem',
'description_problem:ntext',
// 'id_user',
[
    'attribute' => 'id_user',
    'value' => 'user.fio_user',
],
[
    'attribute' => 'id_category',
    'value' => 'category.name_category',
],
//'id_category',
'status_problem',
//'photo_before',
//'photo_after',
```

Добавим две кнопки в модели ActionColumn для реализации операции «Отклонить» и «Решить». Параметр template – вывод кнопок, а параметр buttons – массив с функциями.

```
'template' => '{cancel} {solve}',
'buttons' => [
    'cancel' => function($url, $model) {
        return 'qwer';
    },
    'solve' => function($url, $model) {
        return 'qwer';
    },
],
```

Теперь в ActionColumns отображаются две созданные функции.

Showing **1-2** of **2** items.

#	Временная метка	Название	Описание	ФИО пользователя	Категория	Статус	
1	2026-01-01 18:00:28	Мусор	Мусорные контейнеры переполнились, и жители стали бросать мусор рядом, вонище редкосное, разбиритесь пж	Иванов Иван Иванович	ЖКХ	Новая	qwer qwer
2	2026-01-01 17:53:20	Ямы на дороге	Емае, машина упала в яму!!!:(	Бета Пользователь	Дороги	Решена	qwer qwer

Но эти две функции должны выполняться в случае, если заявки имеют статус «Новая».

```
if($model->status_problem == 'Новая') {
    return 'qwer';
}
```

Теперь вместо заглушки функция будет возвращать html-код ссылку на страницу cancel.

```
if($model->status_problem == 'Новая') {
    return Html::a('Отклонить', ['/problem/cancel', 'id_problem' => $model->id_problem]);
}
```

Таким образом в действиях реализована ссылка Отклонить. Самостоятельно сделайте ссылку на страницу solve – Решить.

Отклонить
qwer

Скопируем файл ProblemCreateForm подкаталога models, и переименуем в ProblemCancelForm.php.

ProblemCancelForm.php

```
class ProblemCancelForm extends Problem
```

Закомментируем правила валидации оставив только поле для причины, которое обязательно для записи.

```
public function rules()
{
    return [
        // [['photo_before', 'photo_after'], 'default', 'value' => null],
        // [['status_problem'], 'default', 'value' => 'Новая'],
        [['reason_problem'], 'required'],
        // [['description_problem', 'status_problem'], 'string'],
        // [['timestamp_problem'], 'safe'],
        // [['photo_before'], 'file', 'extensions' => 'png, jpg, jpeg, bmp',
        // [['id_user', 'id_category'], 'integer'],
        // [['name_problem', 'photo_before', 'photo_after'], 'string', 'max'
        // [['id_category'], 'exist', 'skipOnError' => true, 'targetClass' =>
        // [['id_user'], 'exist', 'skipOnError' => true, 'targetClass' => Use
    ];
}
```

В файле ProblemController скопируем функцию actionUpdate и переименуем в actionCancel.

```php
public function actionCancel($id_problem)
{
    $model = $this->findModel($id_problem);

    if ($this->request->isPost && $model->load($this->request->post()) && $model->save()) {
        return $this->redirect(['view', 'id_problem' => $model->id_problem]);
    }

    return $this->render('update', [
        'model' => $model,
    ]);
}
```

По переданному идентификатору ($id_problem) получаем соответствующий экземпляр модели ProblemCancelForm.

```php
$model = ProblemCancelForm::findOne($id_problem);
```

Отобразим представление (шаблон) cancel.php, куда передадим саму модель, в которой будет расположена форма подтверждения отмены заявки.

```php
return $this->render('cancel', [
    'model' => $model,
]);
```

Проверяем, пришел ли запрос методом POST и успешно ли была выполнена загрузка данных в модель.

Сохраняем данные и меняем статус проблемы – Отклонена.

```php
if ($this->request->isPost && $model->load($this->request->post()) ) {
    $model->status_problem = 'Отклонена';
    $model->save();
    return $this->redirect(['index']);
}
```

Импорт модели ProblemCancelForm.

```php
use app\models\ProblemCancelForm;
```

Скопируем файл create.php подкаталога problem и переименуем его в cancel.php.

cancel.php

Скопируем содержимое файла _form.php подкаталога в файл cancel.php.

```php
<?php

use yii\helpers\Html;
use yii\widgets\ActiveForm;

/** @var yii\web\View $this */
/** @var app\models\Problem $model */

$this->title = 'Отклонить';
$this->params['breadcrumbs'][] = ['label' => 'Problems', 'url' => ['index']];
$this->params['breadcrumbs'][] = $this->title;
?>
<div class="problem-create">

    <h1><?= Html::encode($this->title) ?></h1>

    <div class="problem-form">

    <?php $form = ActiveForm::begin(); ?>

    <?= $form->field($model, 'name_problem')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'description_problem')->textarea(['rows' => 6]) ?>

    <?= $form->field($model, 'timestamp_problem')->textInput() ?>

    <?= $form->field($model, 'id_user')->textInput() ?>

    <?= $form->field($model, 'id_category')->textInput() ?>

    <?= $form->field($model, 'status_problem')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'photo_before')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'photo_after')->textInput(['maxlength' => true]) ?>

    <div class="form-group">
        <?= Html::submitButton('Save', ['class' => 'btn btn-success']) ?>
    </div>

    <?php ActiveForm::end(); ?>

</div>

</div>
```

Оставим только одно поле reason_problem с типом данных полноценный текст.

```php
<?= $form->field($model, 'reason_problem')->textarea(['rows' => 6]) ?>
```

Таким образом реализован интерфейс с формой отклонить заявку.

Отклонить

Причина отклонения

Save

Отклоним любую заявку.

Отклонить

Причина отклонения

Не хочу

Save

Статус поменялся на Отклонена.

Showing **1-2** of **2** items.

#	Временная метка	Название	Описание	ФИО пользователя	Категория	Статус
1	2026-01-01 18:00:28	Мусор	Мусорные контейнеры переполнились, и жители стали бросать мусор рядом, вонище редкосное, разбиритесь пж	Иванов Иван Иванович	ЖКХ	Отклонена
2	2026-01-01 17:53:20	Ямы на дороге	Емае, машина упала в яму!!!:(	Бета Пользователь	Дороги	Решена

В базе данных вручную поменяем заявку на Новая и удалим причину.

←T→	id_problem	name_problem	description_problem	timestamp_problem	id_user	id_category	status_problem	photo_before	photo_after	reason_problem
	1	Ямы на дороге	Емае, машина упала в яму!!!:(	2026-01-01 17:53:20	1	1	Решена	1-1.jpg	1-2.jpg	NULL
	2	Мусор	Мусорные контейнеры переполнились, и жители стали ...	2026-01-01 18:00:28	2	2	Новая	2-1.jpg	NULL	NULL

Скопируем файл ProblemCreateForm подкаталога models, и переименуем в ProblemSolveForm.php.

ProblemSolveForm.php

```
class ProblemSolveForm extends Problem
```

Закомментируем правила валидации оставив только поле с загрузкой файла, которое обязательно для записи.

```php
class ProblemSolveForm extends Problem
{



    /**
     * {@inheritdoc}
     */
    public function rules()
    {
        return [
            // [['photo_before', 'photo_after'], 'default', 'value' => null],
            // [['status_problem'], 'default', 'value' => 'Новая'],
            [['photo_after'], 'required'],
            // [['description_problem', 'status_problem'], 'string'],
            // [['timestamp_problem'], 'safe'],
            [['photo_after'], 'file', 'extensions' => 'png, jpg, jpeg, bmp', 'maxSize' => 10*1024*1024],
            // [['id_user', 'id_category'], 'integer'],
            // [['name_problem', 'photo_before', 'photo_after'], 'string', 'max' => 255],
            // [['id_category'], 'exist', 'skipOnError' => true, 'targetClass' => Category::class, 'targetAt
            // [['id_user'], 'exist', 'skipOnError' => true, 'targetClass' => User::class, 'targetAttribute'
        ];
    }



}
```

Скопируем файл cancel.php подкаталога problem и переименуем его в solve.php.

solve.php

На странице будет присутствовать только поле с загрузкой файла.

```php
<?= $form->field($model, 'photo_after')->fileInput() ?>
```

В модели ActiveForm пропишем специальный MIME-тип, обозначающий, что форма будет содержать бинарные данные, в данном случае загружать файл.

```php
<?php $form = ActiveForm::begin(['options' => ['enctype' => 'multipart/form-data']]); ?>
```

Из файла LkController скопируем функцию actionCreate и вставим в файл ProblemController.

Поменяем модель ProblemCreateForm на модель ProblemSolveForm.

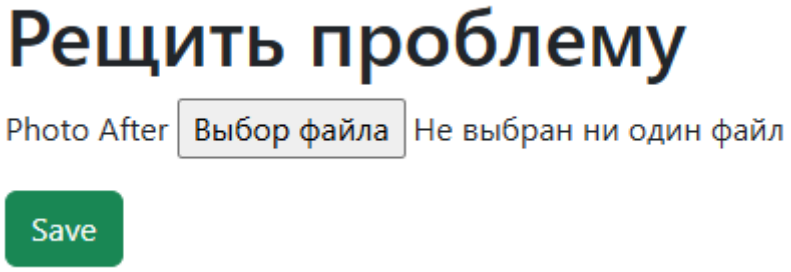
Вместо photo_before пропишем photo_after.

И добавим сохранение заявки со статусом – Решена.

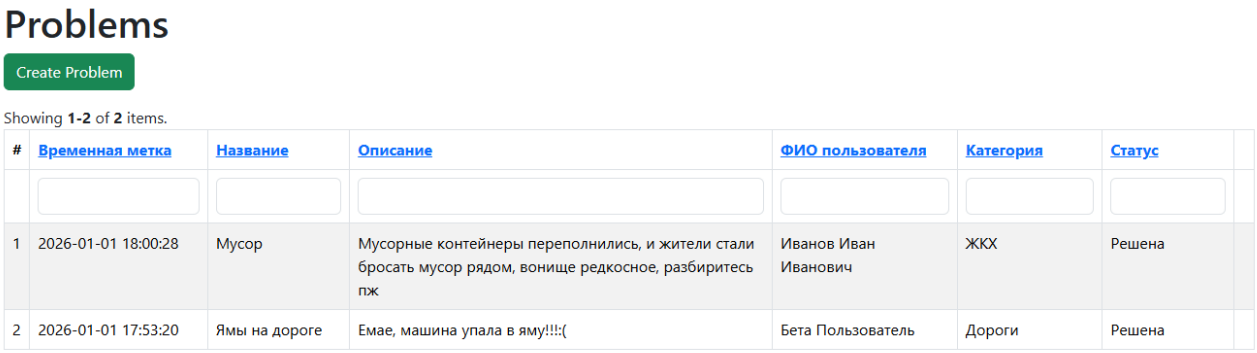
Возвращаем администратора на страницу /problem.

```php
public function actionSolve($id_problem)
{
    $model = ProblemSolveForm::findOne($id_problem);


    if ($this->request->isPost) {
        if ($model->load($this->request->post())) {
            $model->photo_after = UploadedFile::getInstance($model, 'photo_after');
            $newFileName = md5($model->photo_after->baseName . '.' . $model->photo_after->extension . time());
            $model->photo_after->saveAs('@app/web/uploads/' . $newFileName  . '.' . $model->photo_after->extension);
            $model->photo_after = $newFileName;
            $model->status_problem = 'Решена';
            $model->save();
            return $this->redirect(['/problem']);
        }
    }

    return $this->render('solve', [
        'model' => $model,
    ]);
}
```

Интерфейс реализован попробуем загрузить файл.



Статус заявки поменялся на Решена.



Problems

Create Problem

Showing **1-2** of **2** items.

#	Временная метка	Название	Описание	ФИО пользователя	Категория	Статус
1	2026-01-01 18:00:28	Мусор	Мусорные контейнеры переполнились, и жители стали бросать мусор рядом, вонище редкосное, разбиритесь пж	Иванов Иван Иванович	ЖКХ	Решена
2	2026-01-01 17:53:20	Ямы на дороге	Емае, машина упала в яму!!!:(	Бета Пользователь	Дороги	Решена

На главной странице отображается две решенных заявке и фотография.

Мусор

Мусорные контейнеры переполнились, и жители стали бросать мусор рядом, вонище редкосное, разбиритесь пж

Ямы на дороге

Емае, машина упала в яму!!!:(